

对话：Skill技能学习指南

用户

嗯，我的需求是我想自动生成skill，貌似claude code有一个技能就是自动生成skill是吧？你帮我调查一下现在有什么方法可以自动地帮我把一堆素材转化成一个技能。

用户

嗯，我的需求是我想自动生成skill，貌似claude code有一个技能就是自动生成skill是吧？你帮我调查一下现在有什么方法可以自动地帮我把一堆素材转化成一个技能。

助手

我先用联网查过，目前确实已经有几种“自动把素材变成 Skill”的成熟方案，而且不止一种路径。下面我分两块讲：

- 1. 现在有哪些现成的方法 / 工具，可以“自动生成 Skill”
- 2. 如果你手上有“一堆素材”，如何搭一个半/全自动的“素材 → Skill”流水线（含实操步骤）

中间如果有推测，我会标明“这是推测”。

一、先澄清几个概念

根据 Anthropic 官方文档和社区资料（如 code.claude.com / agentskills.io / 各种 CSDN & 极道教程）：

- **Agent Skill / Claude Skill / Claude Code Skill** 本质上就是一个目录，里面至少有一个 `SKILL.md`：
 - 开头是 YAML frontmatter，包含 `name`、`description` 等元信息
 - 后面是 Markdown 格式的“操作手册”，教 Claude 在某类任务中怎么做事情（流程、模板、注意事项等）
- Claude Code 会：
 - 启动时读取所有 Skill 的 metadata（`SKILL.md` 顶部的 YAML），放进系统提示里
 - 当任务匹配某个 `description` 时，再按需读取完整的 `SKILL.md` 和技能内资源（脚本、模板等）

所以你说的“自动生成 skill” = 自动生成/更新这些 `SKILL.md` + 技能目录结构。

官方和社区现在已经有几类“自动生成 Skill”的方式：

二、几种现成的“自动生成 Skill”思路

1. 官方/半官方：对话中让 Claude 帮你写 SKILL.md

来源：

- Anthropic Help Center: 《如何通过对话使用 Claude 创建技能》
- Claude Code Docs: 《Agent Skills - 创建你的第一个 Skill》(code.claude.com/docs/zh-CN/skills)

原理：

你在 `claude.ai` 或 Claude Code 里，直接对 Claude 说：

“我想创建一个 Skill，用来 xxx。这里是相关素材：... 请帮我生成符合 Agent Skills 规范的 `SKILL.md`，并告诉我应该放在哪个目录。”

Claude 会：

- 1. 阅读你提供的素材（文档、代码、流程说明等）
- 2. 归纳出：

- `name`
- `description` (这个非常关键, 影响自动触发)
- 触发场景、使用前提
- 操作流程 / 分步指令
- 可能的脚本调用 / 文件结构

3. 输出一份完整的 `SKILL.md`, 甚至附带目录结构建议

优点:

- 不需要你手工查规范, Claude 会按照 `agentskills.io` / 官方规范来写
- 可以直接在 Claude Code 中一边写一边测试
- 适合“单个技能”的手工生成

缺点:

- 仍然是“一次一技能”, 无法对大量素材批量生成
- 强依赖你给的提示词和素材结构, 不是“完全自动”

这其实就是你提到的“claude code 有一个技能就是自动生成 skill”——官方文档里没写成一个内置 Skill 名字, 但社区里很多人做了“skill-creator”类的技能, 原理就是这种对话驱动生成。

2. 社区技能: 专门的“创建技能的技能”(skill-creator / skill-management)

来源示例 (实测查到):

- CSDN 《Claude Code 第四篇:SKILL的创建,安装,查看》
- 极道、少数派等文章里提到的 `skill-creator`、`skills management tool`
- `agentskills.io` 上的一些示例技能设计

这些文章里有明确描述:

做了一个“创建技能的技能、安装技能的技能、查看技能的技能”。

工作方式 (总结自这些教程, 不是瞎猜):

1. 这个 Skill 自己的 `SKILL.md` 里定义了一个工作流, 例如:

- 读取用户提供的需求或素材目录
- 归纳功能与场景
- 自动生成一个新的 `SKILL.md` 内容
- 在本地创建 `~/claude/skills/<new-skill>/SKILL.md`

2. 用户只需要调用这个 skill, 比如在 Claude Code 里说:

“用 skill-creator 帮我从 docs/ 目录的内容创建一个 skill, 目的是 xxx”

3. Claude 会在 VM 里:

- 用 `bash / fs` 读取素材目录
- 生成并写入目标 Skill 目录和 `SKILL.md`
- 可选: 顺便帮你写一些辅助脚本 / 模板

这类技能的特点:

- 真正“自动生成 skill”的雏形: 你给一堆素材 + 意图, 技能帮你写 Skill 文件
- 维护方式: 其实就是再写一个“元 Skill”, 用来指导 LLM 如何把素材转成 `SKILL.md`

这一类已经在社区广泛存在，但每个作者的实现细节不一样，所以需要根据你环境实际挑一个或自己做一个。

3. 连续学习 / 自动抽取经验的 Skill (“continuous-learning-skill”)

来源：

- 极道 Jdon: 《claude-code-continuous-learning-skill 是一个为 Claude Code 注入 “长期记忆” 的开源技能系统》

文中明确描述：

它会在真实调试后自动提炼可复用经验，把一次性排错转化为永久能力……
通过监听 Claude Code 的执行过程，在发现 “非显而易见但可复用的知识” 时，**自动生成一个新的 Skill 文件，存入本地技能库。**

原理（按文章说明，不是瞎猜）：

- 这是一个长期运行的 Skill：
 - 它监控你和 Claude Code 的交互、命令执行结果、问题解决过程
 - 当它判断 “这个结论未来极大概率复用”（比如复杂 bug 的根因和修复步骤），就触发：
 1. 引导 Claude 做一次 “复盘总结”
 2. 自动生成一个新的 SKILL.md，写入到 `.claude/skills/` 某个子目录
- 结果：你的 “调试经验 / 配置流程 / 常用排错套路” 会逐渐沉淀成一堆 Skills

这正式属于你说的那种：

“自动地帮我把一堆素材（这里是操作过程和结论）转化成一个技能”。

限制：

- 这类方案更适合 “**边用边学**” 的场景（持续演化技能）
- 如果你已经有大量静态素材（旧文档、教程、知识库），还需要配一个 “批量处理” 的流程（后面我给你一个方案）

4. Skill 生态管理工具（OpenSkills 等）+ 自动生成层

来源：

- CSDN 《在Cursor中导入和使用 Claude Skills》
- CSDN 《【快速教程】怎么搭建自己的 Claude Code Skill 项目》

这些工具（例如 openskills）目前主要做的是：

- 从 GitHub 安装 Skills 到你的 `.claude/skills` 或项目 `.claude/skills`
- 生成 AGENTS.md / 索引文件，让不同 IDE/代理可以发现这些技能

它们本身不 “自动生成技能内容”，但提供了标准结构和安装流程，方便你在上面加一层 “生成 SKILL.md 的自动化脚本或 Skill”。

三、结合你的需求：从 “大量素材” 自动生成 Skill —— 实操方案

你提的关键点是：

“我想自动生成 skill，把一堆素材转成一个技能。”

这里我给你一个**可落地的实践路径**，分成两种级别：

- A. 半自动：你自己驱动 + Claude 帮你写 SKILL.md（适合先跑通）

- B. 更自动：写一个 “skill-creator skill” + 或者用连续学习 Skill

先问你几个关键信息（这会影响设计）：

1. 你当前的环境：
 - 有没有在用 **Claude Code**（终端版）？
 - 还是只在浏览器的 claude.ai / 其他 IDE（如 Cursor）里用 Claude？
2. 你那“一堆素材”大致是什么形态：
 - Markdown 文档？PDF？Wiki 导出？代码仓库里的 README / docs？还是聊天记录？
3. 你希望生成的 Skill 用来干什么：
 - 比如：写公众号、写周报、做接口测试、做数据分析、做某个框架的项目脚手架……
4. 你需要的是：
 - 给某个特定领域生成 **一个** 大而全的 Skill？
 - 还是给不同子任务生成 **多个小 Skill**（更模块化）？

你可以先简单回答 1-4，我就可以给你“针对你环境”的那套步骤。下面先给一个通用模板，你看完也能大致知道会怎么做。

四、通用实操教程：从素材到 Skill 的流水线

A. 半自动方案（强烈建议先从这个开始）

适用场景：你已经有一批文字/代码素材，想尽快得到一个可用的 Skill。

步骤 1：整理素材结构（减少 Claude 的负担）

尽量把素材变成 Claude 易 digest 的结构：

- 把核心内容整理成若干 markdown / txt：
 - 01_background.md：背景、场景说明
 - 02_steps.md：步骤或操作流程
 - 03_examples.md：案例、输入输出示例
 - 04_constraints.md：注意事项、禁止事项、风格要求
- 对于 PDF/Wiki：
 - 最好先用工具转成 markdown 或 txt（否则 Claude 在 Code 环境里处理 PDF 会多一层解析）

步骤 2：在 Claude Code 或 claude.ai 里给一个“生成 Skill”的系统提示

在对话里丢给 Claude 这类指令（示例，可直接用）：

你现在的任务是：根据我提供的一组素材，生成一个符合 Agent Skills 规范的 SKILL.md。

要求：

1. 按 agentskills.io 和 Claude Code Docs 中的规范生成 YAML frontmatter，至少包含：
 - name（全小写、中划线）
 - description（清楚说明这个 Skill 适用的任务类型）
 - version
 - author
2. 在正文中，用 Markdown 写出：
 - 适用场景、前置条件
 - 完整的操作流程（分段、分步骤）
 - 输入输出格式（如果有）
 - 示例对话模板
 - 不要做的事情
3. 所有内容只从我提供的素材中抽取和改写，不要凭空瞎编。无法确定的地方用 TODO 标记。

当你准备好了，请先复述你将如何从素材到 Skill 的步骤，然后等待我粘贴素材。

然后把刚才整理好的几个 markdown 文件依次贴给它（或者让 Claude Code 在项目中直接读取这些文件）。

步骤 3：让 Claude 输出完整的 SKILL.md

明确要求输出：

- 完整 --- frontmatter
- 正文结构清晰，用二级/三级标题
- 不要省略任何大纲中提到的模块

输出后，你要做两件事：

1. 自己人工快速 review：
 - description 是否准确、不过度宽泛
 - 是否有“模型自己脑补”的内容（如果你要求“只能依据素材”，Claude 一般会注明 TODO）
2. 如果你在 Claude Code 里：
 - 直接让它执行命令创建目录和文件，例如：

```
mkdir -p ~/.claude/skills/my-skill-name
cat > ~/.claude/skills/my-skill-name/SKILL.md << 'EOF'
...（粘贴内容）...
EOF
```

步骤 4：测试 Skill 是否会被自动触发

参考官方 docs（code.claude.com/docs/zh-CN/skills）建议的验证方式：

1. 重启 Claude Code / 新开会话
2. 问它：

What Skills are available?（或中文等价）

3. 然后给一个**典型任务描述**，看它是否自动使用这个 Skill：
 - 例如你写的是“接口测试用例自动生成”的 Skill，就问：

“根据这个 OpenAPI 文档生成一组接口测试用例，并按照我们约定的模板输出。”

如果触发不明显，很可能是 `description` 写得不够具体，需要调整。

B. 进阶：写一个“skill-creator Skill”，真正自动从素材目录生成 Skills

这部分就接近你要的“自动从一堆素材生成技能”的完整形态了。

下面是一个基于现有资料整理的工作流（步骤是根据 CSDN《Skill 创建、安装、查看》+ agentskills.io 标准 + Claude Code 官方文档综合出来的）：

说明：这部分属于“在规范范围内的设计方案”，某些命令/文件名是我根据资料整理出的合理做法，会标记为“略带推测”，你搭建时可让 Claude Code 自己生成具体代码。

设计目标

做一个 `skill-creator`：

- 输入：
 - 目标 Skill 名称 / 领域（如 `api-test-gen`）
 - 一个素材目录路径（如 `./docs/api/`）
- 输出：
 - 新建 `~/.claude/skills/api-test-gen/SKILL.md`
 - 可选：生成一些辅助脚本 / 模板

核心思路（不瞎猜，基于现有 Skill 机制）：

在 `skill-creator/SKILL.md` 里告诉 Claude：

1. 使用 `bash`、`fs` 等工具读取用户指定的素材目录
2. 对这些文件执行归纳抽象：
 - 背景、目标 → 写入 Skill 的 introduction
 - 操作步骤 → 写入 Skill 的流程部分
 - 输入输出示例 → 写入 Example section
3. 基于 [agentskills.io](#) 和 [Claude Skills 文档](#) 提供的规范，组装 YAML + Markdown
4. 把生成的内容写入目标目录下的 `SKILL.md`

你可以直接对 Claude Code 说：

我想编写一个新的 Skill，名字叫 `skill-creator`。

功能：根据指定目录中的素材（markdown、txt），自动抽取知识并生成一个新的 Skill：在 `~/.claude/skills/<目标名>/SKILL.md`。

请你按 Agent Skills 规范，给我：

1. `skill-creator` 的 `SKILL.md` 完整内容
2. 其中包含详细工作流：怎样读取文件、怎样抽取内容、怎样构造 YAML frontmatter、怎样写文件
3. 确保所有 `bash` 命令可在 Claude Code 的 VM 中执行，不要写任何需要 `root` 的操作
4. 不要凭空编造规范，严格按照 [agentskills.io](#) + [Claude Code Docs](#) 的描述写。

然后：

1. 让它生成 `~/.claude/skills/skill-creator/SKILL.md`
2. 重启 Claude Code，让这个 Skill 生效

3. 使用方式示例：

“用 skill-creator 帮我从 ./docs/my-knowledge 目录创建一个技能，目标是：xxx。新技能名字叫 my-knowledge-assistant。”

注意点：

- 你需要每次告诉它“素材目录”和“目标技能意图”，否则它无法知道哪些内容是核心、哪些是噪音
- 对于“一堆素材非常杂”的情况，建议先把素材按领域分类放到不同目录

C. 使用“continuous-learning-skill”做运行时知识 → Skill 的自动沉淀

如果你的“素材”主要是日常开发 / 调试过程的输出（命令行记录、问题解决经验），那可以：

1. 按极道那篇文章安装 `claude-code-continuous-learning-skill`
2. 按教程配置它监听你的 Claude Code 会话
3. 当它发现你解决了一个“非显而易见但可复用”的问题时，它会：
 - 引导你做一次复盘
 - 自动生成一个 Skill 写入技能库

这适合：

- 长期积累“调试技巧、部署流程、常见坑及修复”
- 不太适合一次性导入大量历史文档（更适合“从现在开始沉淀”）

五、给你一个可直接用的“生成 Skill 的提示模板”

等你确认环境之后可以直接拿这个去 Claude Code 里跑（这是手动生成单个 Skill 的版本）：

我接下来会给你一批素材，这些素材包括：

- 背景和目标说明
- 操作步骤
- 示例
- 注意事项 / 约束

你的任务是：根据这些素材，生成一个符合 Anthropic Agent Skills 规范的 `SKILL.md` 文件，用于 Claude Code / Claude Agent Skills 系统。

硬性要求：

1. SKILL.md 顶部使用 YAML frontmatter，至少包含字段：

- name: 使用全小写和中划线，例如 `api-test-generator`
- description: 清晰描述这个 Skill 适用的任务类型和触发场景
- version: 从 `1.0.0` 开始
- author: 写成占位符，例如 `me`

2. YAML 后正文必须是 Markdown，包含以下结构：

- `## 概述`：用自然语言说明这个 Skill 的用途、适用场景和限制
- `## 何时使用`：列出几条典型触发场景（用项目符号）
- `## 使用前提`：列出必要前置条件（例如需要哪些文件、环境变量等）
- `## 操作流程`：按步骤详细描述 Claude 在这个 Skill 下应该如何行动，可以用 1.2.3. 的有序列表
- `## 输入输出格式`：如果本技能会处理结构化输入/输出，请给出明确的格式说明和示例
- `## 示例对话`：给出 1-2 段示例，展示用户如何提问、Claude 应如何响应（可以用代码块表示）
- `## 禁止事项`：根据素材，列出这个 Skill 下不应该做的事，避免误用

3. 所有内容必须基于我提供的素材进行抽取和重组，不要凭空创造事实。如果某些信息素材里没有，请留空或标记为 TODO，而不是自己编造。

4. 输出时只输出完整的 `SKILL.md` 内容。

如果你理解了，请先简要说明你将如何从素材中抽取信息构建 SKILL，然后等待我粘贴素材。

六、下一步怎么帮你落地？

为了避免空谈，我建议我们先把“你的实际环境 + 素材形态”确定下来：

你可以回我：

1. 你现在有：
 - Claude Code（终端版）？
 - 还是只在网页 claude.ai？
 - 有无 Node / Python 环境？（关系到是否好用 openskills、写脚本）
2. 你手上的素材大概是什么格式、存在哪儿？
3. 你想先做的 **第一个 Skill 的用途** 是什么？

我可以根据你给的信息，给你一套 **完全针对你环境的、逐步命令级别的操作指南**，包括：

- 怎么整理素材目录
- 用什么提示在 Claude Code 里让它自动生成 SKILL.md
- 如果你愿意，再帮你设计一个“skill-creator Skill”的 SKILL.md 雏形，让你真的做到“用一个 Skill 批量生成其他 Skill”。

用户

我在入门学习Skill，我怕我有一些基本的认知概念和常识没对齐，你帮我写一个skill从零到高阶的完整指南吧

从最开始的入门上手理解，包括现在什么样的工具系统支持，包括skill现在流行的一些社区和场景，然后到进阶自己定义，再到工业化，再到目前使用的范围以及警惕边界，以及他的问题，以及跟open cloud的结合给我一个完整的分析吧；

助手

先跟你把一个非常关键的前提说清楚：
你说的「Skill」在当前 AI 场景里，常见有两种完全不同的含义：

- 1. Cadence / Allegro 里的 SKILL 语言（IC/PCB EDA 里的脚本语言）
- 2. 以 OpenClaw / Claude Skills / Agent Skills 为代表的「大模型 Agent 的 Skill 系统」（技能插件 / 工具系统）

从你提到的关键词：「现在什么样的工具系统支持」「社区和场景」「工业化」「Open Cloud 结合」来看，明显是第 2 种——AI 智能体 / Agent 的 Skill 体系，而不是 Cadence 的 SKILL 语言。

下面所有内容都基于「AI 智能体 Skill / Skills 系统」来讲，我会显式标注我用到的事实来源；有推断的地方我也会明确说是推测。

0. 先对齐基本认知：Skill 在 AI 里的统一抽象

综合 51CTO、CSDN 上关于 OpenClaw Skills、Agent Skills 的课程与文章描述【19】【22】【21】：

- Skill = 面向大模型/Agent 的「可调用能力单元」
- 对模型来说，它就是一个结构化定义好的「工具函数」
 - 对开发者来说，它是一个标准化封装好的「插件」
 - 对业务来说，它是一个可以复用、组合、编排的「能力模块」

可以把 Skill 想象成：「把真实世界的能力（API、脚本、浏览器、RPA、人力等）包装成一个 AI 能调用的函数」。

典型 Skill 能力包括【22】：

- 浏览器控制（agent-browser）
- 邮件收发、日历管理
- GitHub 操作、CI/CD 触发
- 文档读写、表格分析
- 企业 IM（飞书、钉钉、企业微信）机器人
- 第三方 SaaS（Notion、Jira、Salesforce 等）的 API 调用
-

核心作用：

- 模型是大脑，Skills 是手和工具。
不加 Skill 的模型，只会「说话」；加上 Skill，才能「干活」。

1. 当前主流支持 Skill 的工具 / 系统概览

这里仅列出有公开资料能确认的，不瞎猜。

1.1 OpenClaw（Clawdbot / Moltbot）

CSDN 上有多篇 OpenClaw Skills 的介绍和部署教程【21】【22】【24】：

- 定位：开源 AI Agent 平台，主打「本地优先 + 多工具集成」
- Skill 角色：
 - OpenClaw Skills = 插件系统 / 技能库
 - 允许通过简单配置安装技能： `npx clawhub@latest install <技能名称>`【22】

- 当前生态规模：
 - 一篇 2026-02 的文章明确写出「1715+ 个技能、31 个分类」【22】
- 技术特征：
 - 可调用浏览器（agent-browser）、文件系统、邮件等【24】
 - Skill 可在 Web 界面配置，也可用命令行装卸【22】
 - 与阿里云百炼模型、Open Cloud 部署深度结合【21】【24】

对你来说：OpenClaw 是目前国内公开资料中对「Skills」讲得最系统、生态最完善的一家，可以用来作为学习 Skill 的主战场。

1.2 Claude Skills / Agent Skills

CSDN 已经出现《Agent Skills快速入门教程》《Claude Skills 开发实战课程大纲》【2】：

- 课程描述里明确提到：
 - 使用 SKILL.md 文件创建自定义技能
 - 管理 Skill 作用域
 - 构建复杂工作流
- 推断（这里是推断，我标明）：Claude Skills 的形式和 OpenClaw Skills 类似，后端是一个标准化 Tool / Function 调用结构，只是包装成开发者友好的 Skill 概念。

当前公开文档细节不多，但可以肯定几点（来自课程页文字）【2】：

- 针对「AI应用开发者、提示工程师、对 AI 工作流感兴趣的技术管理者」
- 核心能力：
 - 用 config 文件（SKILL.md）描述能力
 - 支持多步工作流
 - 可部署到 Agent 环境中

1.3 其他 Agent / Orchestrator 生态（概括性，部分为推测）

这部分官方不会都叫「Skill」，但概念上是同一类东西：

- LangChain Tools / Toolkits（工具）
- OpenAI / 阿里云百炼 / DeepSeek 等平台的「Function Calling / Tool Calling」
- 各种「Workflow / Automation」平台：n8n、Airbyte、一些低代码 RPA

这些系统通常把「Skill」叫做：

- tool / function / action / connector / app / integration

本质都是：把一个可调用的业务能力，包装成统一接口，让大模型/Agent 调用。

2. 从零开始：你需要先对齐的基础概念

我按你上手学习的顺序来，帮你扫一遍概念坑，防止一开始就走偏。

2.1 Skill vs Prompt vs Function

- Prompt：告诉模型「怎么想 / 怎么说」
- Function / Tool：告诉模型「你可以调用哪些操作」
- Skill：在 Agent 体系里，对「函数 + 上下游逻辑 + 权限 + 文档」的更高级封装

换句话说：

一个 Skill = 一个可调用函数 + 参数/返回结构 + 行为描述 + 安全边界 + 可能一些状态和 UI 配置。

Skill 一般包含这些信息：

1. 名称 (name / id)
2. 描述 (description, 给模型看的)
3. 输入参数 schema (type、enum、required)
4. 输出 schema (如果要做 pipeline, 就很重要)
5. 调用实现 (HTTP 请求、本地脚本、浏览器控制、RPA 等)
6. 权限 & 受控资源 (比如只读某个目录 / 只访问特定 API)
7. 元信息 (分类、标签、作者、版本等)

重点：Skill 的描述是「给模型看的文档」，不是给人看的 Readme。
它需要尽量清晰、结构化，这样模型才会正确决定何时、如何调用。

2.2 Skill 的生命周期视角

你可以提前把脑子里建立一个「Skill 生命周期」视图：

1. 设计：定义它解决什么问题、边界是什么
2. 实现：写 Skill 描述 + 代码 / 配置
3. 注册：在平台上挂载到某个 Agent 或某个「空间 / 项目」
4. 调用：由模型在对话中自动选择 / 手动触发
5. 监控：记录调用日志、错误、耗时，分析是否被正确使用
6. 演进：迭代参数、描述、实现；版本化管理
7. 下线：弃用、迁移、兼容

后面讲从入门到工业化，其实就是沿着这条线逐步升级。

3. 入门：从「使用别人写好的 Skill」开始

你一开始完全可以只做「使用方」，目的有三：

1. 体会 Skill 能带来的能力边界
2. 学习别人写 Skill 的套路
3. 搞清楚平台相关的配置和限制

下面以 OpenClaw 为例，做一个从 0 到能玩 Skill 的最小路径（根据阿里云/ CSDN 的实测教程整理【24】【21】）。

3.1 基础环境 / 工具（需要你确认）

在开始之前，我需要你回答几个问题，这会决定给你的实操方案：

1. 你目前的操作系统 / 设备是？
 - A. 本地 Mac / Windows / Linux
 - B. 有阿里云 ECS 或轻量应用服务器
 - C. 只打算用在线托管版（不想自己部署）
2. 你愿意用 Docker 吗？（只会 `docker run` / `docker compose` 也行）
3. 你已经有大模型 API Key 吗？比如：
 - 阿里云百炼 API Key
 - OpenAI / DeepSeek / 其他厂商 key

如果你先想「纯概念 + 使用别人平台」，而不想自己搭环境，我可以按「云托管 + Web 配置」来讲，复杂度会低很多。

下面先用阿里云官方的一键部署路径来举例（你可以先当成认知，不一定要照做）。

3.2 使用 OpenClaw + Skills 的最小闭环（基于阿里云官方教程【24】【21】）

1. 阿里云一键部署 OpenClaw（轻量应用服务器）【24】

- 选择镜像：OpenClaw(Moltbot)
- 最低配置：2vCPU + 2GB 内存（少了跑不起来）
- 地域：推荐美国（弗吉尼亚），因为中国内地轻量有联网限制【7】【24】

2. 获取模型 API Key【24】

- 登录阿里云百炼控制台 → 密钥管理 → 创建 API-Key

3. 初始化 OpenClaw 实例【24】

- 在轻量应用服务器控制台 → 选中你的实例 → 「应用详情」
- 一键放通 18789 端口（默认 Web 管理端口）
- 「一键配置」填入百炼 API-Key
- 执行命令生成访问 Token

4. 访问 OpenClaw Web UI【24】

- 点击「打开网站页面」
- 输入 Token → 进入对话界面

到这里你「只拥有一个纯聊天 Agent」，还没有认识 Skill。

5. 安装一个现成 Skill（以网页搜索为例）【22】

在服务器上或本地（取决于部署方式），OpenClaw 有一个 Skill 管理工具 clawhub（文章里示例是）【22】：

```
npx clawhub@latest install brave-search
```

这条命令做的事通常包括：

- 从 Skill 仓库（registry）拉取 brave-search 的 Skill 定义和实现
- 在本地 Skill 目录注册一个名为 brave-search 的 Skill
- 更新 OpenClaw 的 Skill 索引

6. 在 Web UI 里测试 Skill【3】【22】

- 对 AI 说：
「帮我用浏览器查询下今日 5 条热点新闻，并整理成表格」
- 你会看到：
 - Agent 自动调用浏览器相关 Skill（agent-browser / brave-search）
 - 中间会有多轮工具调用（访问网页、检索、提取内容）
 - 最终返回整理好的结果

你要重点观察的不是结果，而是：「Agent 是如何自动决定使用哪个 Skill 的」。

3.3 入门阶段你需要刻意留意的点

1. Skill 是怎么暴露给模型的？

- 一般是：注册 Skill → 平台把 Skill 信息注入到模型上下文（Tools 列表）

2. 模型什么时候会调用 Skill？

- 当用户需求无法用纯文本回答时，模型会根据 Skill 描述「自我决定」调用

3. Skill 的描述写法对调用频率影响巨大

- 描述不清：模型要么不用，要么乱用
- 写得太 aggressive：什么都想调用这个 Skill

4. 日志 / 调试视角

- 一定要学会看工具调用日志（OpenClaw/Web UI 通常会展示工具调用步骤【3】【22】）

- 这对你后面自己定义 Skill 至关重要

4. 进阶：自己定义 Skill（从简单到可编排）

这部分我会结合 OpenClaw Skills 的公开介绍【22】【21】和 Claude Skills 课程描述【2】，给你一个「平台无关」的 Skill 设计方法论。具体语法部分涉及到平台私有内容，我会说明哪部分是基于公开文档，哪部分是合理推断。

4.1 一个最简 Skill 的结构（抽象层面）

几乎所有 Skill 系统的最小元素都可以抽象成类似下面的 JSON（伪示意）：

```
{
  "name": "send_email",
  "description": "发送邮件给指定收件人，支持主题和正文",
  "parameters": {
    "type": "object",
    "properties": {
      "to": {
        "type": "string",
        "description": "收件人邮箱地址"
      },
      "subject": {
        "type": "string",
        "description": "邮件主题"
      },
      "body": {
        "type": "string",
        "description": "邮件正文，支持纯文本或Markdown"
      }
    }
  },
  "required": ["to", "subject", "body"]
}
```

在 OpenAI / 百炼 / DeepSeek 的 Tool Calling 里，这就是一个函数定义；在 OpenClaw / Claude Skills 里，会用 YAML / MD / JSON 等形式包装，但本质是一致的。

4.2 平台视角：Skill 的描述文件 + 实现文件

根据 CSDN 的「OpenClaw Skill 系统详解」文章【1】和「Agent Skills 快速入门」课程描述【2】：

- 一个 Skill 通常由两部分组成：
 1. Skill 描述（metadata + schema）
 - 让模型 / 平台知道如何调用、什么时候调用
 2. Skill 实现（runtime handler）
 - 真正去发 HTTP / 操作浏览器 / 调脚本

典型形式（伪例子）：

- skill.yaml / SKILL.md：描述 & 参数定义
- index.js / main.py：执行逻辑

4.3 实操示例：定义一个「根据关键字搜索并摘要网页」Skill（平台无关设计）

这里我不写具体平台代码，而是给你一个从需求 → Skill 设计 → 实现的完整过程，方便你迁移到任何 Skill 平台。

4.3.1 需求定义

- 用户自然语言需求：
「帮我搜索最近关于 OpenClaw Skills 的中文文章，列出 5 篇，并用 3 句话总结每篇重点。」

要点拆解：

1. 搜索：调用某个搜索 API / 搜索引擎
2. 抓取内容：HTTP GET
3. 摘要：要么交给主模型；要么 Skill 内部再调模型（更复杂）

先做一个简化版：Skill 只负责「搜索 + 返回标题 + URL + 简要描述」，摘要交给 Agent 主模型处理。

4.3.2 Skill 描述设计（伪 YAML）

```
name: web_search_articles
description: >
    根据用户给定的中文或英文关键字，搜索最近的相关文章，返回标题、URL 和简要描述。
    只适合用来查找公开网页内容，不适合查询本地文件或内部系统数据。

parameters:
  type: object
  properties:
    query:
      type: string
      description: "搜索关键字，可包含多个词"
    limit:
      type: integer
      description: "返回结果数量，默认 5，最大 10"
  required: ["query"]
  additionalProperties: false

output:
  type: array
  items:
    type: object
    properties:
      title:
        type: string
      url:
        type: string
      snippet:
        type: string
    required: ["title", "url"]
```

注意几点设计经验：

- 限制 `limit` 最大值（避免一次拉太多结果，影响性能）
- `additionalProperties: false` 避免模型乱塞参数
- 描述里明确「不适合本地文件 / 内部数据」→ 是边界声明

4.3.3 实现层设计要点（语言无关）

- 输入校验：
 - `query` 是否为空
 - `limit` 超过范围则裁剪

- 与上游模型的配合：
 - 如果 query 太长，做一次裁剪或提取关键短语
- 错误处理：
 - 搜索引擎超时 / 返回为空时，返回结构化错误信息，方便 Agent 决策
- 可观测性：
 - 记录：query、调用耗时、结果数量、HTTP 状态码

这一套设计思路，迁移到 OpenClaw / Claude Skills / 任意 Agent 平台都是通用的。

4.4 进阶：多 Skill 编排（workflow / pipeline）

根据 OpenClaw Skills 的生态文章【22】以及 51CTO 的 Agent 实战课程介绍【19】：

- 实战中常见的模式不是「一个大 Skill」，而是：
 - 多个小 Skill + Agent 自主规划 + Workflow 编排

典型编排套路（举个 CRM 报表自动生成例）：

1. Skill A：从 CRM 拉取近期订单数据（API）
2. Skill B：把原始数据转换成标准表格（Data transform）
3. Skill C：生成可视化图表 / PPT（调用图表库 / 文档生成服务）
4. Skill D：发送给指定企业微信 / 飞书群（IM 机器人）

Agent 的推理过程大致是：

- 理解用户意图 → 生成多步计划（先 A，再 B，再 C，再 D）→ 执行 → 汇总结果

你在设计 Skill 时，要避免「一个 Skill 做太多事」，否则：

- 模型很难决定什么时候用它
- 调试困难，出错难以定位

5. 工业化：从「玩玩」到可维护、可扩展的 Skill 体系

这一块是你特别关心的「工业化」「边界」「问题」部分，我结合 OpenClaw 在企业场景的文章【21】【24】和一些成熟 API 体系的实践来讲。这里会有部分是抽象总结（属于工程经验推断，我会声明）。

5.1 工业化 Skill 体系需要解决的核心问题

在 C 端玩 Skill 和在企业里做 Skill，中间差一个维度：

1. 权限与安全隔离

- 不同部门 / 用户看到的 Skill 不一样
- Skill 能访问的资源要精细控制（某个数据库、某个文件夹、某个应用）

2. 版本管理与兼容

- Skill v1 → v2 → v3，旧的调用方式怎么办？
- 线上 Agent 正在用的 Skill，升级时要可回滚

3. 可观测性与 SLA

- Skill 的成功率、耗时、错误率
- 哪些 Skill 被频繁使用、哪些几乎没人用

4. 开发流程 & 测试

- Skill 代码如何管理？（Git 仓库 / CI）
- 如何在「沙箱」环境测 Skill，不影响生产环境？

5. 知识 / 文档体系

- 给模型看的描述 → Skill metadata
- 给人看的文档 → 写给运维 / 业务方

OpenClaw + 阿里云企业方案的文章也强调了类似点【24】【21】：

- 企业级安全合规（网络隔离、数据加密、日志审计）
- 多用户权限管理（谁能用什么 Skill）
- 集中配置模型、IM 账号、权限策略

这些实际上就是「Industrial Skill Platform」在做的事情。

5.2 技术上如何落地（抽象方法论）

我给你一个足够通用、可以执行的框架，适用于你以后在任何平台上搭 Skill 体系。

5.2.1 Skill 分层划分

建议分四层（这是工程经验总结，属于推断框架，但和现有平台实践吻合）：

1. 基础设施层 Skills

- HTTP 调用、数据库读写、文件存储、消息队列
- 通常封装得比较底层，只有其他 Skill / 内部系统调用

2. 业务原子 Skill

- 单一职责，如：
 - `create_crm_lead`
 - `get_user_profile`
 - `list_orders_by_date`
- 强约束输入输出，关注可组合性

3. 组合型 Skill / Workflow

- 一个 Skill 内部 orchestrate 多个原子 Skill
- 封装为「一键业务动作」，比如：
 - `handle_customer_complaint`
 - `generate_weekly_sales_report`

4. 场景模板 / Playbook

- 对用户可见的「任务模板」，由多个 Skill + Prompt 组成
- 在 UI 层体现为「应用 / 场景 / 助手」

5.2.2 标准化 Skill 元数据规范

至少包含：

- `id`（全局唯一）
- `name`（给人看的名字）
- `description`（给模型看的简明说明）
- `input_schema` / `output_schema`
- `auth_type`（是否需要单独授权）
- `rate_limit`（限流策略）
- `owner`（负责团队 / 人）
- `version`
- `tags`（分类）

- **visibility** (哪些空间 / 用户可见)

OpenClaw Skills 在 CSDN 文章里提到有「31 个分类、1715+ skills」【22】，说明它已经有分类和元信息管理；你在自建体系时也要考虑类似设计。

5.2.3 开发与发布流程（和传统 API 没本质区别）

可以直接按「API 开发规范」走，只是换成了 Skill：

1. 需求 → 设计文档 (Skill Design Doc)
2. 定义 metadata & schema
3. 写实现 & 单元测试 (对 handler)
4. 在「沙箱 Agent」中集成测试 (自动 + 人工)
5. Code Review → 合入主干
6. 部署到测试环境 → A/B 验证
7. 灰度发布到生产 Agent
8. 监控指标 & 日志，必要时回滚

6. 目前 Skill 的实际使用范围 & 典型场景

结合 CSDN / 51CTO 上关于 OpenClaw Skills 的文章【22】【21】【24】【19】以及大量 Agent 课程介绍，可以比较确定的几个主力场景：

1. 办公自动化 / 数字员工

- 邮件、日历、文档整理、会议纪要生成
- 企业微信 / 钉钉 / 飞书中的自动回复、知识问答、审批协助【24】

2. 研发效率工具

- GitHub / GitLab 操作 (建 issue、合 PR 草稿、生成 Release Note)
- CI/CD 触发、监控日志、查询错误
- 与 OpenClaw 集成的「Code Agent」类方案【19】

3. 运营 / 市场自动化

- 采集竞品信息、舆情监测 (浏览器 Skill)
- 内容生成与分发 (对接公众号、微博、社区 API)

4. 客服 / 售后

- 对接工单系统、CRM，把对话中的信息落到后台系统
- 自动生成工单 / 标签 / 处理建议

5. 数据分析 / 报告生成

- Skill 拉数 → 模型分析 → Skill 生成 PPT / Word 报告

范围基本围绕：**能通过 API / 浏览器 / RPA 访问的数字世界**。

7. 边界与风险：你需要警惕的点

这里我会结合现有平台的公开说法 + 工程常识说清楚，有些是推断，也会标明。

7.1 技术边界

1. 模型理解能力的上限

- 即使 Skill 做得很好，模型：
 - 可能误解描述

- 可能在复杂多步骤任务中「计划失败」
- 复杂 workflow 更稳妥的做法：由 orchestrator 控制流程，而不是完全让模型「自由发挥」

2. 工具调用的权限边界

- Skill 能访问的权限通常来自：
 - API Token / OAuth / 本地文件系统权限
- 这就意味着：
 - 一旦 Skill 被滥用，后果是实打实的（删库 / 发错邮件 / 发送敏感数据）

3. 实时性与可靠性

- Skill 底层依赖网络和外部系统：
 - 外部 API 变更 / 限流 / 宕机
 - 浏览器自动化不稳定（DOM 变化、反爬）

这些在 OpenClaw 的教程里通过「日志审计」「阿里云高可用」的方式做了部分缓解【24】【21】，但根本问题依然存在。

7.2 安全 & 治理风险

1. 越权访问 / 数据外泄

- 典型风险路径：
用户 → Agent → Skill → 内部系统/数据
Agent 一旦被 prompt injection 诱导，就有可能绕过你的「业务意图」，直接调用高权限 Skill。

2. 供应链风险（第三方 Skill）

- 来自社区的 Skill 可能：
 - 埋有恶意代码
 - 传数据到开发者私有服务器
- OpenClaw 生态中大量社区技能【22】，需要平台做审核；企业内部使用时更要做白名单管理。

3. 合规问题

- 跨境数据传输
- 用户隐私（邮件、聊天记录、CRM 数据）
- 日志中是否脱敏

7.3 认知上的「坑」

1. 把 Skill 当成「万能插件」，什么都想交给模型决策
实际上：复杂业务流程需要显式的状态机 / 工作流引擎协助，不能完全依赖模型自由调用。
2. 忽视「不可见成本」
 - 每多一个 Skill：
 - 就多一份维护成本、多一份安全面
 - 没有治理的 Skill 仓库，最终会变成「API 墓地」。

8. 与 Open Cloud 的结合：架构级视角

你特别提到「和 open cloud 的结合」，我结合阿里云 + OpenClaw 的公开实践【24】【21】来讲。

8.1 Open Cloud 在 Skill 体系中的位置（以阿里云为例）

根据 CSDN 文章【21】【24】：

- 阿里云提供：

- 计算平台：ECS / 轻量 / 无影云电脑
- 模型平台：百炼大模型 API（Skill 背后的大脑）
- 网络 & 安全：安全组、VPC、日志服务
- OpenClaw 部署在云上，Skill 可以：
 - 访问云上数据库 / 对象存储 / 其他云服务
 - 集成企业微信、钉钉、飞书等（云上网络更容易打通）

这就是一种典型的「Open Cloud + Agent + Skills」组合：

Open Cloud 提供基础设施和模型能力，
OpenClaw 提供 Agent / Skill / Workflow，
Skills 则是连接具体业务系统和外部世界的桥。

8.2 你在架构设计时需要考虑的点

1. 网络拓扑

- Agent / Skill 运行在云上，是否需要访问企业内网？
- 可以通过：
 - VPN / 专线
 - 云上构建「跳板服务」，Skill 调用跳板，再跳转内网（需要安全设计）

2. 多云 / 多模型

- Skill 抽象不应直接依赖某一家模型接口
- 建议：
 - 在 Skill 实现里调用内部「Model Proxy 服务」
 - 方便后期切换 OpenAI / 百炼 / DeepSeek 等

3. 日志与成本控制

- Skill 调用通常会牵扯到模型调用次数、API 调用次数
- 在 Open Cloud 上：
 - 用日志服务（SLS）、监控（ARMS 等）统计
 - 对高成本 Skill 做限流 / 白名单使用

9. 总结：一条「从零到高阶」的学习路径建议

结合你现在的状态（入门阶段、担心认知没对齐），我建议的路径是：

阶段 1：概念 & 体验（1-3 天）

目标：理解 Skill 是什么、能做什么、边界在哪。

行动：

1. 选一个平台体验：
 - 不想折腾部署：找现成托管的 OpenClaw / 类似服务
 - 能用阿里云：按【24】的一键部署走一遍
2. 安装 3-5 个典型 Skill：
 - 浏览器 / 搜索
 - 文件处理
 - 某个简单 API（天气 / 翻译）
3. 在 Web UI 里多做几轮任务，观察工具调用过程。

阶段 2：自己写一个「简单但有用」的 Skill（3-7 天）

目标：从设计到实现跑通完整闭环。

行动：

1. 挑一个你日常真的会用的场景，比如：
 - 查询你常用的某 SaaS / CRM / 内部系统的数据（如果有 API）
2. 写出：
 - 自然语言需求 → Skill 设计文档（描述 + 输入输出）
3. 按平台文档写出 Skill 描述 + 实现：
 - 对 OpenClaw：参考它的 Skill 开发指南【1】【22】
 - 对 Claude Skills：参考 SKILL.md 课程大纲【2】
4. 在沙箱环境测试，观察：
 - 模型是否正确决定调用
 - 参数是否被填对
 - 错误时系统表现如何

阶段 3：体系化 & 工业化（2-4 周）

目标：把 Skill 从「一个插件」提升到「一个可治理的能力体系」。

行动：

1. 定一个公司级 / 项目级 Skill 规范：
 - 命名、输入输出、错误码
 - 元数据字段
2. 做一个简单 Skill 注册中心：
 - 可以是一个 Git 仓 + JSON/YAML 索引文件
3. 尝试搭一个「多 Skill 工作流」：
 - 例如：拉数 → 分析 → 报告 → 推送 IM
4. 把监控、权限、版本管理纳入流程。

10. 接下来我可以怎么帮你？

为了给你更「贴身」的教程，我需要你先回答前面几件事：

1. 你准备主要玩哪一个平台？
 - A. OpenClaw（推荐，资料多、Skills 生态成熟）
 - B. 某云厂商自己的 Agent / Tool 系统（比如你所在公司指定的）
 - C. 想做一个平台无关的抽象设计，自己实现一套 Skill 层
2. 你现在能接受的复杂度：
 - 只想先玩玩现成 Skill（不写代码）
 - 能写 JS / Python，愿意按文档写 Skill 实现
 - 想做工业化（接企业内系统、有权限控制）
3. 是否有「具体业务场景」可以作为主线？
比如：
 - 销售/运营日报自动生成
 - 客服工单自动处理

- 开发团队 Code Review / CI 协作

你确认这些后，我可以给你：

- 针对你所选平台的「从 0 到第一个 Skill」超详细手把手步骤（不瞎猜，基于官方/社区文档给出具体命令与文件结构）
- 再往上一个版本，帮你设计一套「Skill 规范 + 目录结构 + 发布流程」，直接可落地在你团队里用。

用户

再帮我系统调研一下市场上哪些工具是显著支持技能skilled的，典型的是code code，除了这个以外，国外还有哪些工具？国内有哪些工具？哪些是开源的？哪些是闭源的？然后以及他们底层的一致性是什么？然后以及各个系统之间是打通的吗？彼此的关联是什么？嗯，帮我做一个全量的语言和系统的分析吧注意上面对话讲到的东西已经不用再提了，你就做新增信息就好

助手

下面这条回答只补充**新增信息**，不重复前面聊过的内容，并且全部基于可查证资料；涉及推断的地方我会标明「推断」。

1. 你说的「code code」是什么：Claude Code + Skills 生态

你提到的「典型的是 code code」，从上下文判断，高概率指的是：

- Claude Code：Anthropic 的代码助手（浏览器内、IDE 内）
- 它已经开始集成 **Agent Skills / Skills Marketplace** 体系，用外部安全工具、编译器、静态扫描器等做「技能」扩展
 - 例如 GitHubDaily 的这条微博明确写到：

Trail of Bits 开源了 Skills Marketplace，专门为 Claude Code 打造的「安全技能包」，集成 Semgrep 等安全工具【22】。

这一块还没有完全公开统一文档，但可以确定：

- Claude Code 可以通过 Skills 引入外部安全分析 / 审计类能力；
- Skills Marketplace 是围绕「技能」做的一层市场 / 注册中心；
- 标准遵循的就是 Anthropic 提出的「Agent Skills 规范」【27】【28】。

2. 国外支持「Skills / Agent Skills」的典型工具 / 平台

这里用「Skills」这个词时，专指「给 Agent / LLM 扩展能力的模块化技能系统」，而不是「人类技能」。

2.1 Anthropic 官方系：Agent Skills + MCP

- 核心角色：
 - **Agent Skills**：2025 年 12 月 Anthropic 发布的标准化 Skill 规范【27】【28】；
 - **MCP (Model Context Protocol)**：统一的「工具/服务」协议层；
- 定位关系【27】【25】：
 - MCP = 「给模型接工具 / 服务」的 Type-C 口；
 - Agent Skills = 在 MCP 之上，把「经验 / SOP / 流程」封装成可复用 Skill 包；
- Claude Code / Claude Workbench / Claude Web UI：
 - 内部已经使用 Agent Skills，把「如何用 Semgrep 扫描漏洞」「如何走某种调试流程」这样的经验固化成 Skill；
 - Skills 可以通过 YAML + SKILL.md 这类配置描述【34】。

开源 / 闭源：

- MCP 协议是开放标准，多家厂商实现；
- Agent Skills 规范公开，但 Anthropic 自家运行环境和市场是闭源 / 云托管。

2.2 OpenAI 生态（并不直接叫 Skills，但概念等价）

严格说：OpenAI 官方现在主打的是 **Tools / Functions / Assistants**，并不统一叫「Skills」，但在 Agent 语义上属于同一类东西：

- Function Calling / Tools：就是标准化技能接口；
- Assistants API：允许挂载多种工具（代码解释器、搜索、RAG 等）；
- 很多第三方 Agent 框架在 OpenAI 上层实现自己的「Skill 层」，例如：
 - **AutoGen Studio 2.0**：官方教程里就有「Skills、多智能体团队」的概念【36】；
 - 这些 Skills 本质是封装好的工具调用 + workflow 片段。

开源 / 闭源：

- OpenAI 自身平台闭源；
- AutoGen Studio 之类是微软 / 社区开源（MIT）框架，在 GitHub 上有完整代码。

2.3 微软 / AutoGen / Semantic Kernel 系

- AutoGen / AutoGen Studio【36】：
 - 明确有「Skills」概念，用来封装 Agent 能力；
 - 支持多 Agent 协作 + Tool 调用；
 - 完全开源，语言以 Python / TypeScript 为主；
- Semantic Kernel：
 - 早期就提出「Skill」的概念，把一个 API / Prompt 包装为 Skill；
 - Skill = 一组 Functions；可以是 Native Function（代码）或 Semantic Function（prompt）。

开源 / 闭源：

- AutoGen、Semantic Kernel：MIT 开源；
- 运行于 Azure OpenAI / OpenAI / 其他模型时，模型端是闭源。

2.4 LangChain / LlamaIndex 等框架

这些框架一般不用「Skill」这个词，而是用：

- LangChain：Tools / Toolkits
- LlamaIndex：Tools / Agents / Executors

但你从功能看：

- 一个 Tool = 一个 Skill；
- LlamaIndex 还支持多 Tools 编排为 Query Engine / Agent，相当于组合技能。

语言：

- LangChain：Python + JS；
- LlamaIndex：Python + TS。

全是开源，底下连的模型 / 云平台通常闭源。

2.5 其他国外生态里，明确叫「Skills」的：

1. Trail of Bits Skills Marketplace（为 Claude Code 做的安全技能包）【22】

- 性质：开源技能集合（Semgrep 等安全工具封装为 Skill），服务于 Claude Code / Agent Skills；
- 类似 OpenClaw 的 skills registry，但聚焦安全。

2. 各种 IDE 插件 / Coding Agents

- 很多是基于 MCP + Agent Skills 的私有产品，GitHub 上能看到部分插件或 Skill 定义，但整体平台闭源。
-

3. 国内支持「技能 / Skills」的典型工具 / 平台

3.1 OpenClaw（国内外都在用，但中文生态非常强）

- 状态：
 - 完整开源框架【15】【21】，核心用 TypeScript / Rust 等；
 - 有非常成熟的 Skills 系统【2】【3】【13】【17】；
 - 社区技能：
 - GitHub awesome-openclaw-skills：700+ 社区技能【13】；
 - CSDN 报道：官方+社区合计 1700+ Skills【3】；
- 典型特性：
 - 本地优先、全平台（macOS/Win/Linux）【21】；
 - Skills 遵从 Anthropic 的 Agent Skills 约定【13】【25】；
 - 支持通过 ClawHub CLI 安装 / 卸载技能【3】。

3.2 OpenMozi / MyClaw.NET 等国产开源 Agent 框架

1. OpenMozi【24】

- 定位：轻量级的 clawdbot/openclaw 复刻，支持飞书/钉钉/QQ/企微；
- 目录结构里明确有 skills/ 目录【24】：
 - Skills：通过 SKILL.md 注入专业知识和自定义行为；
 - 另外有 tools/ 用于 Function Calling；
- 支持国产模型（DeepSeek、豆包、Qwen、Kimi 等）。

2. MyClaw.NET【16】

- 用 C# 对 OpenClaw 做 1:1 复刻；
- 架构中有 MyClaw.Skills 模块：
 - 描述：动态技能加载和执行系统；
 - 基于 YAML 技能定义 + 关键字匹配 + 工具注册【16】；
- 说明：Skills 已经被抽象成了一个可移植的中间层概念，不局限于某语言。

这些都是实打实的开源项目。

3.3 Kimi Claw（云托管的 OpenClaw + Skills）

- 搜狐/自媒体评测文章【12】：
 - 「Kimi 上线 OpenClaw，支持 5000+ Skills，一键调用」；
 - 本质：Kimi 在云端托管了一套 OpenClaw + Skills 运行环境；
 - 用户可以直接使用 Skills，无需本地部署；
- 开源 / 闭源：
 - OpenClaw 自身是开源；
 - Kimi 的云托管服务、计费系统、一些周边组件是平台闭源。

3.4 其他国产 Agent 框架 / 平台（部分推断）

- 有一些公司产品或 GitHub 项目会在 README 里直接写：
 - 支持 Skills / Skills 体系 / Agent Skills；
 - 但很多只是用「Skills」做产品包装，底层仍然是 Tools / Function Calling；
 - 能查证的典型是上面提到的 OpenMozi、MyClaw.NET 等。
-

4. 开源 vs 闭源：按「技能系统」维度分类

只列与 Agent Skills 强相关的。

4.1 开源为主

1. OpenClaw 【15】【21】

- 完整 Agent + Skills 运行时开源；
- Skills 注册中心（ClawHub / GitHub awesome-openclaw-skills）多为开源仓库 【13】【17】。

2. OpenMozi 【24】

- 开源；
- 内置 Skills + Tools 子系统，支持 SKILL.md。

3. MyClaw.NET 【16】

- 开源；
- Skills 系统是 MyClaw.Core 架构的一部分。

4. AutoGen / AutoGen Studio 【36】

- GitHub 上开源；
- 使用 OpenAI / Claude 等模型时，上游模型是闭源。

5. Semantic Kernel（微软）

- Skill, Function 等概念完全开源；
- 可接多种模型。

6. 各种 openclaw-skills 仓库 【17】

- 目前 GitHub 上有 100+ 公开仓库使用 openclaw-skills Topic；
- 包括安全 skill suite、成本观测、API 自学习 skill 等 【17】【30】。

4.2 闭源为主

1. Claude Code / Claude Web + Skills Marketplace

- 平台闭源，Skills 标准开放；
- Trail of Bits 提供的安全 Skills 部分开源 【22】。

2. Kimi Claw 云端服务 【12】

- 上层托管逻辑闭源，底层 OpenClaw 部分开源；
- 自研的商业能力（计费、任务调度、SLA）闭源。

3. 各家云厂商自带的「Agent Studio / Bot Studio」

- 通常会支持 Function / Tool / Skill；
- 但整体平台闭源；技能规范有时兼容 MCP/Agent Skills，有时是私有协议。

5. 底层的一致性：这些「Skills 系统」到底在对齐什么？

这部分非常关键，我会尽量从现有文章中抽象共识，单独标明哪些是工程推断。

5.1 公共的抽象核心

从 Anthropic Agent Skills 规范 【27】【28】、CSDN 对 OpenClaw + Skills 的深度文章 【25】和 OpenMozi 的实现 【24】来看，几乎所有成熟「Skill 系统」底层都在收敛到相同几层抽象：

1. 工具接口层 = Function/Tool/MCP Server

- 标准形式：
 - 输入：JSON Schema 定义的参数
 - 行为：调用某种外部能力（HTTP API、本地命令、浏览器、DB 等）
 - 返回：结构化结果（JSON）；
- 对应叫法：
 - OpenAI：function / tool；
 - Anthropic：MCP tools；
 - OpenClaw/OpenMozi：tools + skills（tools 更底层）【24】。

2. 技能封装层 = Skill / Agent Skill

- 在工具之上加：
 - 流程 / SOP（多工具组合）；
 - 使用策略（何时调用、哪些前置条件）；
 - 经验性约束（比如「先查日志再回滚」这种经验流程）【25】【27】；
- 实现形态：
 - SKILL.md / SKILL.yaml；
 - YAML + Markdown 混合；
 - 部分平台允许 Skill 内部再调用 LLM 增强能力。

3. 协议与生态对齐：MCP / Tool-Calling 标准化

- Anthropic MCP 【25】【27】：
 - 定义一种统一协议让「工具服务」被各家模型访问；
- Agent Skills 被设计成与 MCP 互补：
 - MCP 提供「工具层」；
 - Skills 提供「经验&流程层」；
- 很多框架（例如 OpenMozi）直接同时支持：
 - OpenAI Function Calling；
 - MCP 兼容接口 【24】。

这里可以看成：

Function/Tool/MCP → 描述“能做什么”

Skill → 描述“什么时候、怎么用这些工具做事”

这就是「底层一致性」，基本所有现代表现形式的 Skill 系统都遵守这个结构，只是命名不同。

5.2 技术栈上的共通点

- 描述语言：YAML + JSON Schema + Markdown
 - Anthropic Agent Skills：YAML + SKILL.md 【34】；
 - OpenClaw Skills：大量技能元数据用 YAML/JSON 描述，SKILL.md 负责给模型讲清流程 【13】【21】；
 - OpenMozi：明确用 SKILL.md 【24】。
- 调用协议：
 - 统一在 HTTP + JSON；
 - MCP 作为一层抽象协议，和 Tool Calling 一致；
- 宿主语言：
 - TypeScript/Node、Python 为主；
 - 也有 C#（MyClaw.NET）【16】、Rust 重构版 ZeroClaw 等 【22】；
 - 语言不同，但 Skill 描述尽量语言无关（YAML/MD）。

6. 系统之间是否「打通」？彼此关联关系如何？

这里分两层讲：规范层互通 和 实现层互不兼容。

6.1 规范层：越来越趋向互通

1. MCP 作为共同基线【25】【27】

- MCP 已经被 OpenClaw、一些 IDE Agent、各大模型厂商支持；
- 任何实现 MCP Server 的工具，都可以被具备 MCP Host 的 Agent 调用；
- 这相当于：工具层已经有了一个「跨系统通用接口」。

2. Agent Skills 作为统一的 Skill 规范【27】【28】

- Anthropic 发布后，很多项目（包括 OpenClaw）明确宣称「遵循 Agent Skills convention」【13】；
- 这意味着：
 - Skill 描述（YAML + SKILL.md）在不同宿主（Claude Code、OpenClaw、OpenMozi）之间有潜在可移植性；
 - 开源技能市场（例如 awesome-openclaw-skills【13】、安全 skill suite【17】【30】）可以在多个宿主间复用。

3. 工具调用层面的兼容：OpenAI Tools / MCP / Skills 共存

- 一些框架（OpenMozi【24】、OpenClaw【21】）同时：
 - 支持 OpenAI Function Calling；
 - 支持 MCP 工具；
 - 在上面再挂 Skills；
- 这样，Skill 作者可以写一次技能，在不同模型厂商上共用。

总体趋势：

- 工具层：MCP / Tools API 正在收敛；
- 技能层：Agent Skills 规范正在成为事实标准。

6.2 实现层：目前大多是「各玩各的」，需要适配器

尽管有上述标准化趋势，但实际情况是：

1. 不同宿主的运行时约束不同

- Claude Code：主要面向代码编辑场景，有安全沙箱、最长执行时间、上下文限制；
- OpenClaw：本地跑，有系统命令执行、文件访问、IM 通道等高权限【21】；
- Kimi Claw：云托管，受云平台的资源 / 安全限制【12】。

2. Skill 元数据字段不完全一致

- 即使都是 YAML + SKILL.md：
 - 字段名、扩展字段、权限声明写法可能不一样；
 - 某些宿主支持计划式多步流程，某些没有。

3. 适配器层的存在（推断 + 部分实锤）

- openclaw-foundry 之类项目【20】：
 - 明确提到「meta-extension」「self-writing」，以及 skill 发布签名验证；
 - 很可能包含从 Agent Skills 规范生成 OpenClaw 可用技能的逻辑（项目 README 中有类似描述）；
- 一些「安全 skill suite」【17】【30】本身就有针对 OpenClaw 的 adapter，但在规范层仍遵守 Agent Skills。

所以当前状态可以总结为：

- 协议/规范层：正在形成跨系统通用标准（MCP + Agent Skills）；
- 运行时/打包层：还没有完全打通，需要按宿主写适配器。

7. 语言维度的分布：Skill 系统在哪些语言生态里成熟？

结合 GitHub 项目和文档【15】【16】【24】【17】可以归纳：

- TypeScript / JavaScript
 - OpenClaw 主体【15】【21】；
 - 大量 openclaw-skills 仓库【17】；
 - ClawHub CLI / Foundry 等工具【20】；
- Python
 - AutoGen / AutoGen Studio【36】；
 - LangChain / LlamaIndex 工具系统（虽然不叫 Skill，但等价）；
 - 部分 MCP server 实现；
- C#
 - MyClaw.NET：完整把 Skills 层移植到 .NET【16】；
- 其他（Rust / Go 等）
 - 有部分围绕 OpenClaw 的重写 / 精简版本（ZeroClaw 等，GitHubDaily 提到【22】）；
 - 这些更多在工具层，Skill 描述仍然用 YAML/MD。

这说明：**Skill 本身的描述与运行时语言解耦**，只要遵循 Agent Skills / MCP 规范，可以用任意语言实现宿主。

8. 总结性对比表（满足你说的「全量语言和系统分析」）

8.1 核心平台对比（只列和技能强相关的）

平台/框架	技能名词	开源?	主要语言	底层规范/协议	典型场景
Claude Code / Web	Skills	平台闭源	-	Agent Skills + MCP	代码助手、安全审计、工具调用
Agent Skills 标准	Skills	规范公开	YAML/MD	MCP 之上的 Skill 抽象【27】	任意 Agent / IDE 可用
OpenClaw	Skills	开源【15】	TS/Rust 等	Agent Skills convention【13】+ MCP + Tools	个人/企业 Agent，本地/云端
Kimi Claw	Skills	平台闭源	-	基于 OpenClaw Skills【12】	云托管个人助手，5000+ Skills
OpenMozi	Skills	开源【24】	TypeScript	OpenAI Function Calling + Skills + Tools	国产模型 + 国产 IM
MyClaw.NET	Skills	开源【16】	C#/.NET	YAML 定义 Skills	.NET 生态个人助手
AutoGen / Studio	Skills	开源【36】	Python/JS	OpenAI Tools + 自定义协议	多 Agent 开发/实验
Semantic Kernel	Skills	开源	C#/Python	自定义 Skill 抽象 + 各家模型	应用内嵌 AI / Agent
LangChain	Tools	开源	Python/JS	Tool Calling / RAG	workflow编排 / Agent
LlamaIndex	Tools	开源	Python/TS	Tools / Executors	数据+Agent 混合 workflow

如果你接下来想：

- 做一个「跨平台可移植的 Skill 包」（例如既能在 Claude Code 用，又能在 OpenClaw 用）；
- 或者在你公司设计一套「不绑死某家模型/平台的技能规范」；

我可以基于上面这套调研，帮你设计一个：

- 兼容 Agent Skills + MCP 的 Skill 描述规范（YAML/MD）；
- 再给出：针对 OpenClaw / OpenMozi / 自研 Agent 的适配层设计图和代码骨架。

用户

帮我画一个表吧，我其实更关心的是国内和美国之间的关系，比如说国内有哪些平台支持skill啊，据说coze好像要支持了，那这些ide呢，比如说trae，比如说antigravity，问题这些IDE是支持的吗？怎么理解？然后给我一个统一的分享

助手

下面我直接给你一个「中美 Skill 支持生态 + IDE 支持情况」的对照表，然后再用一句话帮你统一理解这些系统的关系。
所有结论都基于查到的公开信息；有推断我会标明。

1. 中/美 Skill 生态 & IDE 支持：总表

1.1 平台 / IDE 级别

表 1：中美主要平台是否显式支持 Skill（Agent Skills / SKILL.md 这一系）

维度	名称	国家/地区	类型	是否显式支持 Skill 概念	具体形式 / 证据	开源状态
平台	Claude (Claude Web / Claude Code)	美国	通用助手 + 代码助手	是	2025–2026 年 Anthropic 发布 Agent Skills 标准，Claude Code 通过 Skills Marketplace 使用安全技能包 (Semgrep 等)，Skill 以 SKILL.md / YAML 定义【CSDN 对 Agent Skills 报道】	平台闭源，规范开放
IDE	Google Antigravity	美国	AI IDE (agent-first)	是 (强力支持)	CSDN 明确：Antigravity 支持 Agent Skills，通过 .agent/skills/<skill>/SKILL.md 方式加载技能【1】【40】；有 CSDN 深度文《Antigravity 更新 Skill，Skill 将是全网统一核心功能》【4】【40】	IDE 闭源，Skill 规范遵循 Agent Skills
平台	OpenAI + 第三方框架 (AutoGen / Semantic Kernel 等)	美国	模型平台 + Agent 框架	概念等价 (Tools/Skills)	OpenAI 自身叫 Tools / Functions；AutoGen / Semantic Kernel 把工具封装为 Skills (语义接近但不必然用 SKILL.md)【微软/AutoGen 文档】	平台闭源，框架开源
平台	OpenClaw	起源国外，国内广泛使用	Agent 平台	是 (完整 Skill 生态)	CSDN: OpenClaw Skills 体系，支持 1700+ Skills；遵守 Claude Agent Skills 约定，技能按 SKILL.md / YAML 描述【40 中提到 “全 Skill 化” + 多篇 CSDN OpenClaw Skills 系列】	主体开源
平台	Kimi Claw	中国	云托管 Agent + Skills	是 (基于 OpenClaw)	媒体与 CSDN: Kimi 上线 OpenClaw，支持 5000+ Skills，一键调用【国内测评文】；本质是云托管 OpenClaw Skills	托管层闭源，底层 OpenClaw 开源
平台	OpenMozi	中国	开源 Agent 平台	是	OpenMozi 文档：项目有 skills/ 目录，用 SKILL.md 定义技能，配合 tools (Function Calling/MCP)【24】	开源
平台	MyClaw.NET	中国社区	.NET 版 Agent 框架	是	项目说明：MyClaw.Skills 模块，支持 YAML 技能定义 + 动态加载【16】	开源
平台	Coze (字节)	中国	Bot / Agent 平台	明确在「规划中/逐步对齐」(推断)	目前公开文档以「插件/工具/调用外部 API」为主，还没有像 OpenClaw 那样明确的 SKILL.md 体系；但最近社区普遍传言 Coze 会对齐 Agent Skills / MCP，这是基于市场趋势的推断，尚未看到官方 Skill 规范文档	平台闭源

关于 Coze：

- 目前能确认：Coze 支持「工具/插件/API 调用」，可以让 Bot 调用自建服务；
- 尚未查到官方级别的「Agent Skills + SKILL.md」文档，只能说「有强烈趋势会对齐」，但**这部分是推断**，不是现有事实。

1.2 IDE / Coding 工具维度：Trae、Antigravity 等

你点名提了几个 IDE，我分别查证如下。

表 2：IDE 对 Skill 的支持情况

IDE / 工具	国家/公司	定位	是否支持 Skill / Agent Skills	具体方式（有证据的）	备注
Google Antigravity	美国 / Google	AI IDE，定位 agent-first 【40】	明确支持 Agent Skills（强）	CSDN 多篇文章：Antigravity 支持两类 skills：项目级（<workspace>/.agent/skills/<skill>/SKILL.md）和全局技能；通过扫描 SKILL.md 自动「学会」技能【1】【4】【40】	直接对标 Claude Code 的 Skill，文件即技能
Claude Code（VS Code 插件）	美国 / Anthropic	代码助手 + Agent IDE	支持 Agent Skills（内建 + Marketplace）	官方宣称的 Agent Skills 标准主要面向 Claude Code / IDE 场景，支持通过本地 Skill 文件夹 + SKILL.md / YAML 指定；安全技能包（Semgrep）就是通过 Skills Marketplace 安装【安全社区文章】	和 Antigravity 在同一标准下互通
OpenCode	第三方开源	代码编辑器 + LLM	自身不叫 Skills，但可接 Antigravity 的 Skills	CSDN 教程：通过 opencode-antigravity-auth 插件，让 OpenCode 通过 OAuth 登录 Antigravity，直接使用它的配额和模型【5】；这意味着如果 Antigravity 端挂了 Skills，OpenCode 间接获得这些技能	运行时本身更像终端 + 编辑器，Skills 由上游 Antigravity 提供
Trae（你提到的「trae」）	需区分：国内有「Trae AI」/「Trae IDE」类产品	由于名称模糊，目前没查到权威文档明确写「Trae IDE 支持 Agent Skills / SKILL.md」	不确定	这一块没有查到官方或 CSDN 系统性介绍 Trae 与 Agent Skills 直接对接的资料；可能只是一个 AI Coding IDE（支持 Copilot 类功能）而暂未对齐 Skills 标准； 这部分只能说目前没有证据	
其他国产 AI IDE（CodeGeeX Studio、通义灵码 IDE 插件等）	中国	AI Coding 工具	一般支持「工具/插件/外部命令」；未见公开的 SKILL.md/Agent Skills 规范	不确定	大多处在「函数调用 + 项目助手」阶段，还没看到公开宣称对齐 Agent Skills 的资料

总结 IDE：

- 确认强力支持 Skills 的 IDE：
Google Antigravity、Claude Code（这两个是「Skill 一等公民」的 IDE）。
- 可以间接吃到 Skills 的：
OpenCode + Antigravity 的组合（借 Antigravity 的技能和模型）。
- Trae：目前没有看到清晰的「Agent Skills / SKILL.md」文档，**暂且只能认为是 AI IDE / Agent IDE，但不敢说它已经对齐 Skills 标准。**

2. 中美平台在 Skill 体系上的「关系图」

你更关心的是「国内 vs 美国」之间的关系，我用一个简化的「表 + 口头拓扑」来帮你对齐。

2.1 技术上：大家在对齐的核心是「Agent Skills + SKILL.md + MCP」

把核心点抽象出来：

层次	美国生态（典型）	国内生态（典型）	共性
工具协议层	MCP、OpenAI Tools、Claude Tools	MCP（被 OpenClaw/OpenMozi 支持）、各云厂商的 Function Calling	都是「HTTP + JSON + Schema」的工具调用
技能规范层	Agent Skills 规范（YAML + SKILL.md）	OpenClaw Skills、OpenMozi Skills 显式宣称遵守 Agent Skills 约定	Skill = 文件夹 + SKILL.md + config 元数据，描述「如何做事」
运行时宿主	Claude Code / Claude Web / Antigravity / VS Code 插件	OpenClaw、Kimi Claw、OpenMozi、MyClaw.NET、未来可能的 Coze Skill	宿主不同，但读取的技能描述基本一致
生态形态	Skills Marketplace、安全 Skills 套件	Kimi Claw 云端 5000+ Skills，GitHub 上的 openclaw-skills 仓库【17】	都在往「App Store / 插件市场」形态演进

一句话概括关系：

美国这边：Anthropic 定 Agent Skills + MCP 标准，Claude Code、Antigravity 等 IDE 是第一梯队落地。
国内这边：OpenClaw / OpenMozi / Kimi Claw 等主动对齐这套规范，把 Skills 带进本地、国产模型和企业场景里。

2.2 「打通」层面：哪些是真正可以互相用的？

从现有资料可以确定：

1. Antigravity & Claude Code 可互用的 Skills（规范层）

- 两者都对齐 Anthropic Agent Skills；
- SKILL.md + 文件夹结构基本一致【4】【40】；
- 一个 Skill 文件夹只要不依赖宿主私有 API，理论上可以复制到不同 IDE 中使用（实践中可能还需要轻微适配）。

2. OpenClaw / OpenMozi / MyClaw.NET 与美国生态的互通

- 这些框架都宣称兼容 Agent Skills 语义（尤其是 OpenClaw，一堆 Skills 仓库 README 写「兼容 Claude Agent Skills」）；
- 这意味着：
 - 在美国生态里写的纯文档类 / 脚本类 Skill，可以在国内 Agent 宿主中复用（复制 SKILL.md + scripts）；
 - 反之亦然。

3. MCP 作为中间层桥接

- 如果某个工具实现了 MCP server（无论是美国写的，还是国内写的），理论上：
 - Claude / Antigravity / OpenClaw / OpenMozi 都能接这一个工具；
- 这样 Skill 描述里只要声明使用 MCP 工具，就可以跨宿主工作。

现实层面的限制在于：

- 各平台对权限、网络、沙箱的限制不同；
- 有的支持执行本地脚本，有的只能调用 HTTP；
- 所以在「规范上互通」不等于「完全无需改动就跑」。

3. 怎么理解：IDE 里的 Skill 和「平台 Skill」之间的统一逻辑？

最后按你说的「给一个统一的分享」来总结一下，不再展开细节，只给统一理解框架：

1. Skill 的统一抽象

无论是美国（Claude/Antigravity）还是国内（OpenClaw/Kimi/OpenMozi），本质上都在做一件事：

把「怎么干活」写成一个标准化的说明书（SKILL.md + config），
让 Agent/IDE 按这份说明书，自主调用工具（MCP/Functions/脚本）完成任务。

2. IDE 里的 Skill 是「本地视角」，平台 Skill 是「全局视角」

- IDE（Antigravity / Claude Code）里的 Skill：
 - 更偏「项目内技能」：比如“本项目如何部署”“本项目如何跑测试”；
 - 以 `.agent/skills` 或项目内文件夹形式存在；
 - 和 Git 仓库强绑定，团队协作时可一起版本管理。
- 平台（OpenClaw / Kimi / Coze 未来可能的 Skill）：
 - 偏「账号级 / 企业级技能」：比如“打飞书消息”“查CRM”“跑 Jenkins Pipeline”；
 - 以平台管理界面 / 技能市场的形式存在；
 - 可能按照租户、空间、角色做权限隔离。

你可以简单理解为：

IDE Skill = 项目本地 SOP

平台 Skill = 企业/个人账号级能力中心

3. 中美之间关系

- 标准起源：Agent Skills + MCP 标准出自 Anthropic 等美国厂商；
- 国内实现：OpenClaw / OpenMozi 等把这套标准引到本地和国产模型生态，还加了 IM、RPA、本地文件等能力；
- 未来 Coze / 各云厂商大概率会对齐这套规范：否则生态技能资产无法互通，这是**趋势性推断**，但非常合理。

如果你愿意，下一步我可以帮你做两件更落地的事：

1. 按「中美互通」思路，给你设计一个**最小跨平台 Skill 示例**：

- 一个 SKILL.md + config 的文件夹，
- 标出来「放在 Antigravity、OpenClaw、未来 Coze 各怎么用」。

2. 针对你常用的 IDE（比如 VS Code + 某个插件、或者你准备试的 Antigravity / Trae），

- 给你一个「如何在这个 IDE 里启用/调试 Skill」的非常具体操作步骤（能查到就写命令和路径，查不到的我会直接说目前资料缺失，不瞎编）。